```
  <li>Contact</li>
</ul>
```

Your polyfill fails silently and returns `null` as if nothing were wrong:

```
document.querySelector '.links .active'
# null
```

Suppose, though, that you don't have time to implement (or even read) the entire specification. Change your polyfill to throw an exception if it's used for something that you know it doesn't support:

```
document.querySelector ?= (selector) ->
  if /^#/.test selector
   (document.getElementById (selector.replace /^#/gi, ''))
  else
    throw new Error 'Not supported by this implementation'
```

This polyfill throws an exception if it doesn't implement what you try to use it for.

Now at least other programmers are warned that your polyfill doesn't implement the specification (assuming that they didn't read your tests to discover that already). If needed, they can override your implementation with a more complete one.

It's not just host objects that are inconsistent across browsers, though—some language features aren't implemented by all browsers. In fact, the absence of things like `Array.map` in older browsers was one of the early motivations for creating CoffeeScript.

11.3.2. Polyfilling language features

Each browser moves at a different pace. One browser might implement a new feature of JavaScript while it's still just ideas, whereas another browser may implement the same feature years after it's standardized.

For example, take a language feature that you're familiar with—being able to create a new object with an existing object as the prototype using `Object.create`. It's easy to take this feature for granted, but unfortunately, when you test your program in Browser X, you discover that it isn't defined. What do you do? Polyfill: